Bow Valley College

Course Outline

SODV3301

Software Development Techniques
Fall 2020 - Spring 2021

Last Updated: 8/24/2020 8:25:51 PM

Care has been taken to obtain copyright permission to reproduce this material. Any information that will enable Bow Valley College to obtain copyright clearance for any material not acknowledged would gladly be received by:

Bow Valley College
345 6th Avenue SE
Calgary AB T2G 4V1
Attn: Copyright Officer
email: copyright@bowvalleycollege.ca

© Bow Valley College

SODV3301 Software Development Techniques

COURSE DESCRIPTION

This course introduces learners to fundamental approaches to software development techniques by using algorithms, data structures and their technique to implementation. Learners will explore common technique to find the shortest path and minimal cost path traversal techniques. They will also learn a simple way to compress data and mathematical optimization techniques will reduce the computational time for the software they will built. Learners are also exposed to common technical interview questions and strategies to solve them.

REQUISITES	Complete or concurrently enroll in: • DATA3201 - Database Management (3) • SODV3302 - Systems Development and Object-Oriented Design (3)
EQUIVALENTS	None
CREDITS	3
HOURS	45
ELIGIBLE FOR PLAR	No

COURSE LEARNING OUTCOMES

Bow Valley College is committed to ensuring our graduates can demonstrate their abilities in key areas that will make them effective citizens and encourage their development as lifelong learners. In addition to the discipline-specific skills that learners acquire in their programs, the College has identified ten learning outcomes.

College-Wide Outcomes:

1. Communication
2. Thinking Skills
3. Numeracy and Financial Literacy
4. Working with Others
5. Digital Literacy
6. Positive Attitudes and Behaviours
7. Continuous Learning
8. Health and Wellness Awareness
9. Citizenship and Intercultural Competence
10. Environmental Sustainability

#	COURSE LEARNING OUTCOME(S)	COLLEGE WIDE OUTCOMES SUPPORTED
1	Analyze and determine potential methodologies to address problem types	1, 2, 4, 5
2	Apply greedy programming to solve optimization problems based upon problem characteristics	2, 4, 5
3	Apply dynamic programming to solve optimization problems based upon problem characteristics	2, 4, 5
4	Perform quick text searching operations using the tries technique	2, 4, 5
5	Optimize searches through the use of self-balancing trees	2, 4, 5
6	Compose and trial approximation algorithms to address exponential/non-polynomial problems	2, 4, 5
7	Write and optimize code	2, 4, 5

COURSE MODULES AND SCHEDULE

**Course schedule subject to change, depending on delivery mode and term of study. For exact dates, please consult the Course Offering Information in Brightspace.*

WEEK/HOURS	MODULES
Week 1	Module 01: Complexity Analysis and NP-Completeness NP-Completeness • Understand the overview of the course • Know some explanatory visual publicly available tools • Read the article NP-Completeness • View the video NP-Hard and NP-Complete
Week 2	Module 01: Complexity Analysis and NP-Completeness Linear Time Sorting Algorithms • Limitations of Comparison Based Sorting • Counting Sort • Radix Sort
Week 3	Module 02: The Greedy Algorithms The Greedy principle • Understanding the Greedy principle • Problems that can utilize the greedy principle • Know potential applications of greedy algorithms • Understanding the priority queue
Week 4	Module 02: The Greedy Algorithms The Minimal Spanning Tree • Prim's Algorithm for Minimal Spanning Tree construction • Kruskal's Algorithm for Minimal Spanning Tree construction • Application of Minimal Spanning Trees • How Prim's algorithm differs from Kruskal's algorithm
Week 5	Module 02: The Greedy Algorithms The Single Source Shortest Path Algorithm • The Dijkstra's Algorithm for Single Source Shortest Path • The Requirements of Dijkstra's algorithm • Applications of Shortest Path Algorithms
Week 6	Module 02: The Greedy Algorithms The Huffman Compression Algorithm • Understanding compression algorithms • The Greedy approach to compression algorithms • The Huffman Coded Tree • The Huffman code encoders and decoders • Know potential applications of Compression algorithms

Week 7	Module 03: The Dynamic Programming General Principle of Dynamic Programming • Understand overlapping problems • Identify problems that can be solved using dynamic programming • Know the limitations of dynamic programming • Learn the general principle of Dynamic Programming • Apply Dynamic programming for the sum of subset problem instead of the recursive solution
Week 8	Module 03: The Dynamic Programming The Longest Common Substructure (LCS) Problem • Use Dynamic programming to find the Longest Common Substructure • Use LCS algorithm for solving applications (Substructure matching, DNA analysis, image registration etc.)
Week 9	Reading Week Break
Week 10	Module 03: The Dynamic Programming Use Dynamic Programming for Matrix Chain Multiplication • Know the some practical Applications • Matrix Chain Multiplication (Parallel Programming, GPU optimization, Efficient 3D Animation etc.)
Week 11	Module 4: Efficient Self-Balancing Trees, Approximation Algorithms and Code Optimization Tries and Balancing Trees • Apply Trie-search technique for string prefix matching. • Demonstrate efficient searching with Tries • Trie-Application: Lookup table, Dictionary Search, Text auto complete etc. • Optimize tree height by using Tree Rotation
Week 12	Module 4: Efficient Self-Balancing Trees, Approximation Algorithms and Code Optimization Self-Balancing AVL Trees and Approximation Algorithms • Learn the AVL trees (Construction) • Optimize binary search trees by self-balanced AVL trees • Approximate algorithms for optimizing execution time • Learn approximation factor/approximation ratio
Week 13	Module 4: Efficient Self-Balancing Trees, Approximation Algorithms and Code Optimization Approximation traveling salesman problem (TSP) • Learn the NP-completeness of the TSP problem • Approximate near-optimal solution to TSP problems • Write approximation algorithms for NP-complete problem
Week 14	Module 4: Efficient Self-Balancing Trees, Approximation Algorithms and Code Optimization Source Code Optimization • Learn the architecture independent code optimization techniques • Code for High Performance Systems Development
Week 15	Module 4: Efficient Self-Balancing Trees, Approximation Algorithms and Code Optimization Source Code Optimization • Minimize redundant computation and optimize code

ASSESSMENT

COURSE LEARNING OUTCOME(S)	ASSESSMENT	WEIGHT
1, 2, 3, 4, 5	Quizzes	30%
1, 2, 3, 4, 5	Assignments	40%
1, 2, 3, 4, 5	Learning activities	30%

Important: For details on each assignment and exam, please see the Course Offering Information.

PERFORMANCE STANDARDS

A minimum grade of D is required to pass this course. However, a program may require a higher grade in this course to progress in the program or to meet specific program completion requirements.

Please consult with the program area or contact the program chair for further details. A minimum Grade Point Average of 2.0 is required for graduation.

GRADING SCHEME

Grade	Percentage	Grade Point	Description
A+	95-100	4.0	Exceptional: superior knowledge of subject matter
A	90-94	4.0	Excellent: outstanding knowledge of subject matter
A-	85-89	3.67	
B+	80-84	3.33	
B	75-79	3.0	Very Good: knowledge of subject matter generally mastered
B-	70-74	2.67	
C+	67-69	2.33	
C	64-66	2.0	Satisfactory/Acceptable: knowledge of subject matter adequately mastered
C-	60-63	1.67	

D+	57-59	1.33	
D	50-56	1.0	Minimal Pass
F	Less than 50	0.0	Fail: an unsatisfactory performance

REQUIRED LEARNING RESOURCES

Additional learning resources may be found in the Course Offering Information or in Brightspace.

ADDITIONAL INFORMATION

Optional Resource:
Corman, T., Lieserson, C.E. & Rivest,R.L. (2009) *Introduction to algorithms*, (3rd ed.). (The MIT Press).MIT Press. **ISBN: 9780262033848**

Additional information may be found in the Course Offering Information or in Brightspace.

ACADEMIC ACCOMMODATIONS

Learners with a disability (learning, physical, and/or mental health) may qualify for academic and exam accommodations. For more information, or to apply for accommodations, learners should make an appointment with Accessibility Services in the Learner Success Services (LSS) Department. Accessibility Services can also assist learners who may be struggling with learning but do not have a formal diagnosis. To make an appointment visit LSS on the first floor of the south campus or call 403-410-1440. It is the learner's responsibility to contact Accessibility Services and request academic accommodations. For more information, please visit our website at http://www.bowvalleycollege.ca/accessibility.

INSTITUTIONAL POLICIES

Bow Valley College is committed to the highest standards of academic integrity and honesty. Learners are urged to become familiar with and uphold the following policies: Academic Honesty (500-1-7), Learner Code of Conduct, Procedures and Guidelines (500-1-1), Learner Appeals (500-1-12), Attendance (500-1-10), Grading (500-1-6), Academic Continuance and Graduation (500-1-5), and Electronic Communications (300-2-13). Audio or video recording of lectures, labs, seminars, or any other teaching and learning environment by learners is allowed only with consent of the instructor as part of an approved accommodation plan.

Recorded material is to be used solely for personal study and is not being used or distributed without prior written consent from the instructor.

Turnitin:

Students may be required to submit their course work to Turnitin, a third-party service provider engaged by BVC. Turnitin identifies plagiarism by checking databases of electronic books and articles, archived webpages, and previously submitted student papers. Students acknowledge that any course work or essays submitted to Turnitin will be included as source documents in the Turnitin.com reference database, where it will be used solely to detect plagiarism. The terms that apply to a student's use of Turnitin are described on Turnitin.com.

Online Exam Proctoring:

Examinations for this course may require proctoring through an online proctoring service. Online proctoring enables online exam taking within a controlled and monitored environment, thereby enhancing academic integrity. Online proctoring may occur through a variety of methods, including but not limited to:

a. live online proctoring where a remote invigilator authenticates identity and observes completion of an exam using specialized software and recordings;
b. automated proctoring where the exam session is recorded and AI (artificial intelligence) analyzed;
c. browser lockdown that limits access to other applications, websites, copying, printing, screen capture and other functions; or
d. a combination of both live/automated proctoring and browser lockdown.

Course instructors will review recordings, analyses, and data obtained through online proctoring for academic integrity infractions. It is the student's responsibility to meet the technical, software, location, and identity verification requirements necessary to enable online proctoring.

Further details of these policies are available in the Academic Calendar and on the Bow Valley College website, bowvalleycollege.ca.

Learners are encouraged to keep a copy of this course outline for future reference.